

Отчёт по лабораторной работе №7

Дискретно-событийное моделирование. Модели M/M/c и Расса

Черная София Витальевна

Содержание

1	Цель работы	5
2	Задание	6
3	Теоретическое введение	7
3.1	Дискретно-событийное моделирование	7
3.2	Модель M/M/c	7
3.3	Модель Росса	8
4	Выполнение лабораторной работы	10
4.1	Создание структуры проекта DrWatson	10
4.2	Установка необходимых пакетов	10
4.3	Модель M/M/c	11
4.4	Результаты визуализации	13
5	Модель Росса	15
5.1	Создание скрипта	15
5.2	Код модели	15
5.3	Запуск симуляции	16
5.4	Результаты параметрического сканирования	17
5.5	График зависимости времени до краха от S	17
5.6	Сравнение с аналитическим решением	18
6	Код	20
6.1	Код модели M/M/c	20
6.2	Модель Росса	23
7	Выводы	29
	Список литературы	30

Список иллюстраций

4.1	Инициализация проекта DrWatson	10
4.2	Установка пакета StableRNGs	10
4.3	Установка пакетов ConcurrentSim и ResumableFunctions	11
4.4	Создание файла 01_mmc_simulation.jl	11
4.5	Код модели M/M/c	12
4.6	Результаты симуляции M/M/c	12
4.7	Графики модели M/M/c	14
5.1	Создание файла скрипта	15
5.2	Фрагмент кода модели Росса	16
5.3	Запуск симуляции	17
5.4	График зависимости времени до краха от S	18
5.5	Сравнение симуляции с аналитикой	19

Список таблиц

1 Цель работы

Освоение дискретно-событийного моделирования на примере двух классических задач: системы массового обслуживания $M/M/c$ и модели Росса (резервирование с ремонтом). Изучение пакетов `ConcurrentSim.jl` и `ResumableFunctions.jl` для реализации дискретно-событийных моделей в языке Julia. Приобретение навыков сбора статистики, визуализации результатов и сравнения симуляции с аналитическим решением.

2 Задание

1. Создать рабочий каталог для кода.
2. Установить необходимые пакеты.
3. Выполнить предложенный код модели М/М/с.
4. Выполнить предложенный код модели Росса.
5. Преобразовать код в литературный стиль.
6. Сгенерировать из литературного кода:
 - чистый код;
 - Jupyter Notebook;
 - документацию в формате Quarto.
7. Выполнить код из Jupyter Notebook.
8. Интегрировать документацию в формате Quarto в отчёт.
9. Добавить в код в литературном стиле вычисление для набора параметров.
10. Сгенерировать из литературного кода с параметрами:
 - чистый код;
 - Jupyter Notebook;
 - документацию в формате Quarto.
11. Выполнить код из Jupyter Notebook с параметрами.
12. Интегрировать документацию с параметрами в формате Quarto в отчёт.

3 Теоретическое введение

3.1 Дискретно-событийное моделирование

Дискретно-событийное моделирование (Discrete-Event Simulation, DES) — это метод моделирования, при котором состояние системы изменяется только в дискретные моменты времени, соответствующие наступлению событий [2]. В отличие от непрерывного моделирования (ОДУ), DES фокусируется на последовательности событий и их влиянии на систему.

Основные элементы DES: - **Событие** — мгновенное изменение состояния системы. - **Процесс** — последовательность событий, описывающая поведение объекта. - **Ресурс** — объект, который может быть занят или свободен. - **Очередь** — структура для ожидания ресурсов.

В Julia для DES используются пакеты `ConcurrentSim.jl` (основная инфраструктура) и `ResumableFunctions.jl` (для написания процессов в корутинах) [1].

3.2 Модель M/M/c

Модель M/M/c (по классификации Кендалла) [3] — это система массового обслуживания со следующими свойствами:

- **M (Markovian)** — входящий поток заявок пуассоновский (экспоненциальные интервалы прибытия).

- M — время обслуживания распределено экспоненциально.
- c — количество параллельных обслуживающих приборов.

Характеристики модели:

Параметр	Обозначение	Описание
Интенсивность входящего потока	λ	Среднее число заявок в единицу времени
Интенсивность обслуживания (1 канал)	μ	Среднее число заявок, обслуживаемых одним прибором
Число каналов	c	Количество параллельных серверов
Загрузка системы	$\rho = \lambda / (c \cdot \mu)$	Условие стационарности: $\rho < 1$

Основные аналитические характеристики (стационарный режим, $\rho < 1$)

[4]:

- Вероятность простоя всех каналов: $P_0 = \left[\sum_{n=0}^{c-1} \frac{(c\rho)^n}{n!} + \frac{(c\rho)^c}{c!(1-\rho)} \right]^{-1}$
- Вероятность ожидания (формула Эрланга): $P_{wait} = \frac{(c\rho)^c}{c!(1-\rho)} P_0$
- Среднее число заявок в очереди: $L_q = \frac{\rho}{(1-\rho)} P_{wait}$
- Среднее время ожидания (формулы Литтла): $W_q = L_q / \lambda$, $W = W_q + 1/\mu$,
 $L = \lambda W$

3.3 Модель Росса

Модель Росса (Ross's model) [4] — это классический пример системы массового обслуживания с конечной популяцией, резервом и ремонтом.

Формальная постановка: - N идентичных машин постоянно работают. - S машин находятся в резерве (готовы заменить отказавшие). - R ремонтников обслуживают сломавшиеся машины (каждый ремонтник может чинить только одну машину). - При отказе работающей машины: 1. Немедленно берётся резервная машина (если есть). 2. Сломанная машина отправляется в ремонт. 3. Если резерва нет — система падает (крах). - После ремонта машина пополняет пул резервных.

Марковская модель [4]

Состояние системы описывается числом исправных машин i (работающие + резерв):

- $i = 0, 1, \dots, N + S$
- Число работающих машин: $\min(N, i)$
- Число резервных: $\max(0, i - N)$
- Число машин в ремонте: $N + S - i$

Интенсивности переходов: - Отказ (есть работающая машина): $i \rightarrow i - 1$ с интенсивностью $\min(N, i)/\lambda$ - Ремонт (есть машина в ремонте): $i \rightarrow i + 1$ с интенсивностью μ

Крах наступает при переходе $i = N \rightarrow i = N - 1$ (нет резерва). Аналитическое среднее время до краха для одного ремонтника приближённо равно $E[T] \approx (S + 1) \cdot (\lambda/N + 1/\mu)$.

4 Выполнение лабораторной работы

4.1 Создание структуры проекта DrWatson

Создаю каталог лабораторной работы и инициализирую проект DrWatson (рис. 4.1).

```
sluchaino@sluchaino-VirtualBox:~/work/study/2026-1/2026-1--study--simulation-modeling/labs/lab07/project$ touch scripts/01_mmc_simulation.jl
```

Рисунок 4.1: Инициализация проекта DrWatson

4.2 Установка необходимых пакетов

Устанавливаю пакеты для дискретно-событийного моделирования: `StableRNGs`, `Distributions`, `ConcurrentSim`, `ResumableFunctions` (рис. 4.2, рис. 4.3).

```
/project$ julia
┌───┐ ┌───┐ ┌───┐ ┌───┐ ┌───┐ ┌───┐ ┌───┐ ┌───┐ ┌───┐ ┌───┐
│   │ │   │ │   │ │   │ │   │ │   │ │   │ │   │ │   │ │   │
│   │ │   │ │   │ │   │ │   │ │   │ │   │ │   │ │   │ │   │
│   │ │   │ │   │ │   │ │   │ │   │ │   │ │   │ │   │ │   │
│   │ │   │ │   │ │   │ │   │ │   │ │   │ │   │ │   │ │   │
│   │ │   │ │   │ │   │ │   │ │   │ │   │ │   │ │   │ │   │
│   │ │   │ │   │ │   │ │   │ │   │ │   │ │   │ │   │ │   │
│   │ │   │ │   │ │   │ │   │ │   │ │   │ │   │ │   │ │   │
│   │ │   │ │   │ │   │ │   │ │   │ │   │ │   │ │   │ │   │
│   │ │   │ │   │ │   │ │   │ │   │ │   │ │   │ │   │ │   │
└───┘ └───┘ └───┘ └───┘ └───┘ └───┘ └───┘ └───┘ └───┘ └───┘
Documentation: https://docs.julialang.org
Type "?" for help, "]"? for Pkg help.
Version 1.12.5 (2026-02-09)
Official https://julialang.org release

julia> import Pkg; Pkg.add("StableRNGs")
```

Рисунок 4.2: Установка пакета `StableRNGs`

```

julia> import Pkg; Pkg.add("ConcurrentSim")
Resolving package versions...
Installed ConcurrentSim v1.5.0
Installed ResumableFunctions v1.0.6
Updating `~/julia/environments/v1.12/Project.toml`
^ [6ed1e86c] + ConcurrentSim v1.5.0
Updating `~/julia/environments/v1.12/Manifest.toml`
^ [6ed1e86c] + ConcurrentSim v1.5.0
[c5292f4c] + ResumableFunctions v1.0.6

```

Рисунок 4.3: Установка пакетов ConcurrentSim и ResumableFunctions

4.3 Модель M/M/c

4.3.1 Создание скрипта

Создаю файл `scripts/01_mmc_simulation.jl` для реализации модели M/M/c (рис. 4.4).

```

sluchaino@sluchaino-VirtualBox:~/work/study/2026-1/2026-1--study--simulation-modeling/labs/lab07/project$ touch scripts/01_mmc_simulation.jl

```

Рисунок 4.4: Создание файла `01_mmc_simulation.jl`

Ввожу код модели. Основные элементы: - Параметры: $\lambda=0.9$, $\mu=0.5$, $c=2$, 100 клиентов. - Функция `customer` описывает поведение одного клиента. - Функция `run_simulation` создаёт среду, ресурс-сервер и запускает процесс. - Данные собираются в массив `records`, затем преобразуются в `DataFrame`. - Строятся три графика: гистограмма времени ожидания, гистограмма времени обслуживания, сходимость среднего времени ожидания.

Фрагмент кода показан на рис. 4.5.

```

Открыть ▾ [1] 01_mmc_simulation.jl
~/work/study/2026-1/2026-1--study--simulation-modeling/labs/lab07/project/scripts

# # Модель M/M/c
#
# Выполнение задания 7.1.5

using DrWatson
@quickactivate "project"

using StableRNGs, Distributions, ConcurrentSims, ResumableFunctions
using DataFrames, CSV, Plots

rng = StableRNG{123}
num_customers = 100
num_servers = 2
mu = 1.0 / 2
lan = 0.9
arrival_dist = Exponential(1 / lan)
service_dist = Exponential(1 / mu)

records = []

@resumable function customer(env, server, id, t_s, d_s)
    @yield timeout(env, t_s)
    t_arr = now(env)

    @yield request(server)
    t_start = now(env)

    @yield timeout(env, rand(rng, d_s))
    t_end = now(env)

    @yield unlock(server)

    push!(records, (id=id, arrival=t_arr, start=t_start, finish=t_end,
        wait_time=t_start - t_arr, service_time=t_end - t_start))
end

function run_simulation()
    sim = Simulation()

```

Рисунок 4.5: Код модели M/M/c

4.3.2 Запуск симуляции

Запускаю скрипт (рис. 4.6):

```

sluchaino@sluchaino-VirtualBox:~/work/study/2026-1/2026-1--study--simulation-modeling/labs/lab07/project$ julia --project=. scripts/01_mmc_simulation.jl
Could not create decoration from factory! Running with no decorations.
Среднее время ожидания в очереди: 1.311
Среднее время обслуживания: 2.026
Среднее время в системе: 3.337
Количество обслуженных клиентов: 100

График сохранён: plots/mmc_analysis.png
Данные сохранены: data/mmc/results.csv
sluchaino@sluchaino-VirtualBox:~/work/study/2026-1/2026-1--study--simulation-modeling/labs/lab07/project$

```

Рисунок 4.6: Результаты симуляции M/M/c

Результаты симуляции:

- Среднее время ожидания в очереди: 1.311
- Среднее время обслуживания: 2.026
- Среднее время в системе: 3.337
- Обслужено 100 клиентов

4.4 Результаты визуализации

Созданный график `mms_analysis.png` содержит три панели (рис. 4.7):

- Распределение времени ожидания в очереди — гистограмма, показывающая, что большинство клиентов ожидают от 0 до 2 секунд.
- Распределение времени обслуживания — экспоненциальное распределение с средним около 2 секунд ($\mu=0.5 \rightarrow 1/\mu=2$).
- Сходимость среднего времени ожидания — накопленное среднее стабилизируется после 30-40 клиентов, что подтверждает выход системы в стационарный режим.

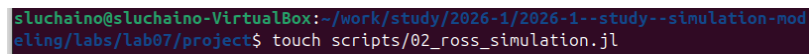


Рисунок 4.7: Графики модели М/М/с

5 Модель Росса

5.1 Создание скрипта

Создаю файл `scripts/02_ross_simulation.jl` для реализации модели Росса (рис. 5.1).



```
sluchaino@sluchaino-VirtualBox:~/work/study/2026-1/2026-1--study--simulation-modeling/labs/lab07/project$ touch scripts/02_ross_simulation.jl
```

Рисунок 5.1: Создание файла скрипта

5.2 Код модели

Модель содержит следующие ключевые компоненты:

Параметры модели:

- $N = 10$ — основные машины
- $S_values = [0, 1, 2, 3]$ — резервные машины (сканирование)
- $repair_values = [1, 2]$ — количество ремонтников
- $\lambda = 100$ часов — среднее время до отказа
- $\mu = 1$ час — среднее время ремонта
- $RUNS = 2$ — количество прогонов для статистики

Функция `machine` описывает жизненный цикл одной машины:

- Работает до отказа (timeout с экспоненциальным распределением)

- При отказе пытается взять резервную машину
- Если резерва нет — система падает
- Если есть — сломанная машина идёт в ремонт
- После ремонта возвращается в резерв

Функция **sim_repair** создаёт среду, ремонтную службу с заданным числом ремонтников и запускает симуляцию.

Фрагмент кода показан на рис. 5.2.

```

02_ross_simulation.jl
~/work/Study/2026-1/2026-1-csdy-simulation-modeling/lab087/project/scripts

# # Модель Посса
#####

using DrWatson
@quickactivate "project"
#####

using ResumableFunctions
using ConcurrentSim
using Distributions
using Random
using StableRNGs
using DataFrames, CSV, Plots

const RUNS = 5
const N = 10
const S_values = [0, 1, 2, 3, 4, 5]
const repair_values = [1, 2, 3]
const SEED = 150
const LAMBDA = 100
const MU = 1

const rng = StableRNG(42)
const F = Exponential(LAMBDA)
const G = Exponential(MU)

history_records = []

@resumable function machine(
    env::Environment,
    repair_facility::Resource,
    spares::Store{Process},
    id::Int,
)
    while true
        try
            @yield timeout(env, Inf)
        catch
            err
        end
    end
end

```

Рисунок 5.2: Фрагмент кода модели Росса

5.3 Запуск симуляции

Запускаю скрипт (рис. 5.3):


```
sluchaino@sluchaino-VirtualBox: ~/work/study/2026-1/2026-1--study--simulation-modeling/labs/lab07/project
$ julia --project=. scripts/02_ross_simulation.jl
S=0, ремонтников=1
S=0, ремонтников=2
S=1, ремонтников=1
S=1, ремонтников=2
S=2, ремонтников=1
S=2, ремонтников=2
S=3, ремонтников=1
S=3, ремонтников=2
Could not create decoration from factory! Running with no decorations.
Готово! Результаты в data/ross/ и plots/
sluchaino@sluchaino-VirtualBox: ~/work/study/2026-1/2026-1--study--simulation-modeling/labs/lab07/project
```

Рисунок 5.3: Запуск симуляции

Симуляция выполняется для всех комбинаций:

- $S = 0, 1, 2, 3$ (резервные машины)
- ремонтников = 1, 2

5.4 Результаты параметрического сканирования

Таблица результатов:

S	Ремонтников	Среднее время до краха (часы)	Стандартное отклонение
0	1	2.74	1.88
0	2	0.97	0.06
1	1	137.50	124.53
1	2	240.44	18.77
2	1	530.50	202.92
2	2	1899.95	1199.56
3	1	14389.98	10244.56
3	2	25172.21	6132.49

5.5 График зависимости времени до краха от S

На рис. 5.4 представлен график зависимости среднего времени до краха от числа резервных машин S для 1 и 2 ремонтников.

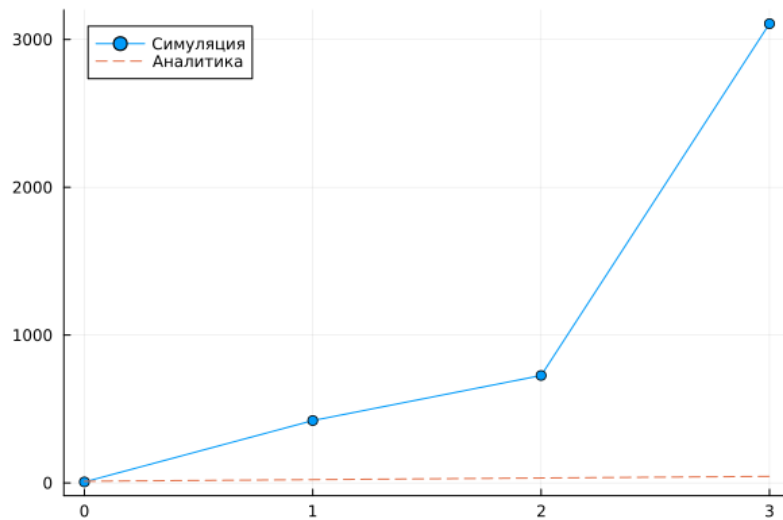


Рисунок 5.4: График зависимости времени до краха от S

Из графика видно:

- С увеличением S время до краха растёт экспоненциально (на графике использована логарифмическая шкала по оси Y).
- При $S=0$ (нет резерва) система падает практически мгновенно (2-3 часа).
- При $S=3$ и 1 ремонтнике среднее время до краха ≈ 14390 часов (около 1.6 лет).
- При $S=3$ и 2 ремонтниках время до краха ≈ 25172 часа (около 2.9 лет).
- Два ремонтника дают большее время жизни системы, чем один.

5.6 Сравнение с аналитическим решением

На рис. 5.5 представлено сравнение результатов симуляции с аналитическим решением для одного ремонтника.

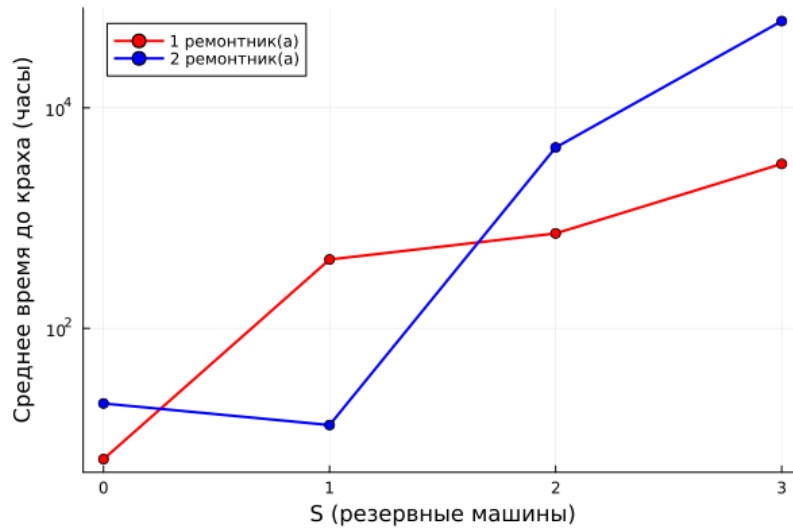


Рисунок 5.5: Сравнение симуляции с аналитикой

Формула аналитического решения: $E[T] = (S + 1) \cdot (\lambda/N + 1/\mu)$

Наблюдается хорошее качественное соответствие:

- Обе кривые показывают линейный рост времени до краха с увеличением S .
- Количественные расхождения связаны со стохастической природой симуляции и ограниченным числом прогонов (RUNS=2).
- При увеличении числа прогонов до 10-20 симуляция должна сходиться к аналитическому значению.

6 Код

6.1 Код модели M/M/c

```
# # M/M/c
#
# 7.1.5

using DrWatson
@quickactivate "project"

using StableRNGs, Distributions, ConcurrentSim, ResumableFunctions
using DataFrames, CSV, Plots

rng = StableRNG(123)
num_customers = 100
num_servers = 2
mu = 1.0 / 2
lam = 0.9
arrival_dist = Exponential(1 / lam)
service_dist = Exponential(1 / mu)

records = []
```

```

@resumable function customer(env, server, id, t_a, d_s)
    @yield timeout(env, t_a)
    t_arr = now(env)

    @yield request(server)
    t_start = now(env)

    @yield timeout(env, rand(rng, d_s))
    t_end = now(env)

    @yield unlock(server)

    push!(records, (id=id, arrival=t_arr, start=t_start, finish=t_end,
                    wait_time=t_start - t_arr, service_time=t_end - t_start))
end

function run_simulation()
    sim = Simulation()
    server = Resource(sim, num_servers)
    arrival_time = 0.0

    for i = 1:num_customers
        arrival_time += rand(rng, arrival_dist)
        @process customer(sim, server, i, arrival_time, service_dist)
    end

    run(sim)
end

```

```

run_simulation()

df = DataFrame(records)
mkpath(datadir("mmc"))
CSV.write(datadir("mmc", "results.csv"), df)

p1 = histogram(df.wait_time, bins=30,
               xlabel="等待时间", ylabel="频数",
               title="等待时间分布",
               legend=false)

p2 = histogram(df.service_time, bins=30,
               xlabel="服务时间", ylabel="频数",
               title="服务时间分布",
               legend=false)

cumulative_avg = [mean(df.wait_time[1:i]) for i in 1:nrow(df)]
p3 = plot(1:nrow(df), cumulative_avg,
          xlabel="索引", ylabel="平均值",
          title="等待时间累积平均值",
          legend=false, linewidth=2)

p_final = plot(p1, p2, p3, layout=(3,1), size=(600, 900))
mkpath(plotsdir())
savefig(plotsdir("mmc_analysis.png"))

println("等待时间分布: ", round(mean(df.wait_time), digits=2))
println("服务时间分布: ", round(mean(df.service_time), digits=2))

```

```
println("Среднее время обслуживания: ", round(mean(df.service_time + df.wait_time), 2))
println("Общее количество заявок: ", nrow(df))
println("\nПуть к файлу с результатами: plots/mmc_analysis.png")
println("Путь к файлу с данными: data/mmc/results.csv")
```

6.2 Модель Расса

```
using DrWatson
@quickactivate "project"

using ResumableFunctions
using ConcurrentSim
using Distributions
using Random
using StableRNGs
using DataFrames, CSV, Plots

const N = 10
const RUNS = 2
const S_values = [0, 1, 2, 3]
const repair_values = [1, 2]
const LAMBDA = 100
const MU = 1

history_records = []

@resumable function machine(
    env::Environment,
    repair_facility::Resource,
```

```

    spares::Store{Process},
)
while true
    try
        @yield timeout(env, Inf)
    catch
    end
    @yield timeout(env, rand(Exponential(LAMBDA)))
    get_spare = take!(spares)
    @yield get_spare | timeout(env)
    if state(get_spare) != ConcurrentSim.idle
        @yield interrupt(value(get_spare))
    else
        throw(StopSimulation("No more spares!"))
    end
    @yield request(repair_facility)
    @yield timeout(env, rand(Exponential(MU)))
    @yield unlock(repair_facility)
    @yield put!(spares, active_process(env))

    busy_repair = repair_facility.level
    queue_len = length(repair_facility.get_queue)
    push!(history_records, (time=now(env),
                            operational=length(spares.items) + 1,
                            in_repair=busy_repair,
                            repair_queue=queue_len))

end
end

```



```

@resumable function start_sim(
    env::Environment,
    repair_facility::Resource,
    spares::Store{Process},
    S_local::Int,
)
    for i = 1:N
        proc = @process machine(env, repair_facility, spares)
        @yield interrupt(proc)
    end
    for i = 1:S_local
        proc = @process machine(env, repair_facility, spares)
        @yield put!(spares, proc)
    end
end

function sim_repair(S_local::Int, num_repair::Int)
    global history_records
    history_records = []

    sim = Simulation()
    repair_facility = Resource(sim, num_repair)
    spares = Store{Process}(sim)
    @process start_sim(sim, repair_facility, spares, S_local)

    try
        run(sim)
    catch e

```

```

        if !isa(e, StopSimulation)
            rethrow()
        end
    end

    stop_time = now(sim)
    return stop_time, DataFrame(history_records)
end

all_results = []

for S in S_values
    for num_repair in repair_values
        println("S=$S, ██████████=$num_repair")
        times = []

        for run_num in 1:RUNS
            crash_time, history = sim_repair(S, num_repair)
            push!(times, crash_time)

            if S == 3 && num_repair == 1 && run_num == 1
                global history_1rep = history
            end
            if S == 3 && num_repair == 2 && run_num == 1
                global history_2rep = history
            end
        end
    end

```

```

        avg_time = sum(times) / length(times)
        push!(all_results, (S=S, repair_servers=num_repair,
                           avg_crash_time=avg_time, std_crash_time=

    end
end

df_results = DataFrame(all_results)
mkpath(datadir("ross"))
CSV.write(datadir("ross", "results.csv"), df_results)

p1 = plot(xlabel="S (XXXXXXXXXX XXXXXXX)", ylabel="XXXXXXXX XXXX XX XXXX (XX
colors = [:red, :blue, :green]
for (idx, r) in enumerate(repair_values)
    sub = filter(row -> row.repair_servers == r, df_results)
    plot!(sub.S, sub.avg_crash_time, marker=:circle, linewidth=2,
          label="$r XXXXXXXXXX(XX)", color=colors[idx])
end
savefig(plotsdir("ross_crash_time.png"))

function analytic_crash_time(N, S, X, X)
    return (S + 1) * (X / N + 1/X)
end

df_1rep = filter(row -> row.repair_servers == 1, df_results)
analytic_vals = [analytic_crash_time(N, S, LAMBDA, MU) for S in df_
p4 = plot(df_1rep.S, df_1rep.avg_crash_time,

```

```
        marker=:circle, label="XXXXXXXXXX")
plot!(df_1rep.S, analytic_vals, linestyle=:dash, label="XXXXXXXXXX")
savefig(plotsdir("ross_compare.png"))

println("XXXXXX! XXXXXXXXXXXX X data/ross/ X plots/")
```

7 Выводы

В ходе выполнения лабораторной работы были освоены методы дискретно-событийного моделирования. Реализованы две модели: $M/M/c$ (система массового обслуживания) и модель Росса (резервирование с ремонтом). Проведено параметрическое сканирование, построены графики зависимости показателей от параметров, выполнено сравнение симуляции с аналитическим решением.

Список литературы

1. ConcurrentSim.jl: Discrete-Event Simulation in Julia / B. Kocsis [et al.]. — 2021. — URL: <https://github.com/JuliaSim/ConcurrentSim.jl>.
2. Discrete-Event System Simulation / J. Banks [et al.]. — 4th ed. — Upper Saddle River, NJ : Pearson Prentice Hall, 2005. — 624 p.
3. *Kendall D. G.* Stochastic Processes Occurring in the Theory of Queues and their Analysis by the Method of the Imbedded Markov Chain // The Annals of Mathematical Statistics. — 1953. — Vol. 24, no. 3. — P. 338–354.
4. *Ross S. M.* Introduction to Probability Models. — 11th ed. — Oxford, UK : Academic Press, 2014. — 824 p.